

The initialisation process may confuse some readers. To understand it, please look at Figures 1 and 2. In Figure 1, you can see how the two functions `do_basic_setup()` and `do_initcalls()`, in turn, invoke other functions. In Figure 2, we take the `sock_init()` function, which is called from `do_basic_setup()`, as an example. Observe how it invokes another set of functions. Because of this, we are unable to classify the functions properly—hence, throughout this section, I will explain the functions as we come across them.

A few other related functions are `netfilter_init` (basic initialisation for `netfilter`) and `rtnetlink_init` (to initialise the routing `netlink` socket interface).

Sockets

Now, let's discuss socket objects. The allocation of socket objects is done by using the following code:

```
struct sock *sk_alloc(int proto_family, int priority, struct proto *prot, int zero_new_sock);
```

You need to specify the protocol family, priority, and the associated struct proto, if you want to 'zero it'.

In the code of the `sk_alloc()` function, `net` corresponds to the Net namespace, and `family` is used to represent the protocol family. You may also note that `priority` is used for proper allocation, and `prot` is used to deal with a new sock instance. If you want to find out how to use `prot`, you can experiment with the following struct (which deals with IPv6):

```
struct proto tcpv6_prot = {
    .name      = "TCPv6",
    .owner     = THIS_MODULE,
    .close     = tcp_close,
    .connect   = tcp_v6_connect,
    .disconnect = tcp_disconnect,
    .accept    = inet_csk_accept,
    .ioctl     = tcp_ioctl,
    .init      = tcp_v6_init_sock,
    .destroy   = tcp_v6_destroy_sock,
    .shutdown  = tcp_shutdown,
    .setsockopt = tcp_setsockopt,
    .getsockopt = tcp_getsockopt,
    .recvmsg   = tcp_recvmsg,
    .backlog_rcv = tcp_v6_do_rcv,
    .hash      = tcp_v6_hash,
    .unhash    = inet_unhash,
    .get_port   = inet_csk_get_port,
    .enter_memory_pressure = tcp_enter_memory_pressure,
    .sockets_allocated = &tcp_sockets_allocated,
    .memory_allocated = &tcp_memory_allocated,
    .memory_pressure = &tcp_memory_pressure,
    .orphan_count   = &tcp_orphan_count,
    .sysctl_mem      = sysctl_tcp_mem,
    .sysctl_wmem     = sysctl_tcp_wmem,
    .sysctl_rmem     = sysctl_tcp_rmem,
    .max_header      = MAX_TCP_HEADER,
    .obj_size         = sizeof(struct tcp6_sock),
```

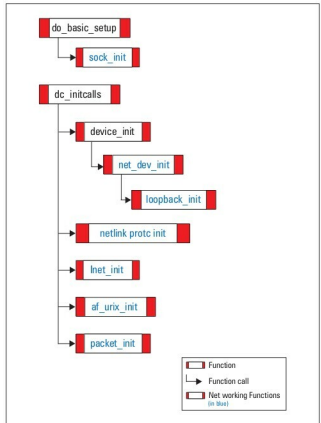


Figure 1: Initialisation of subsystem

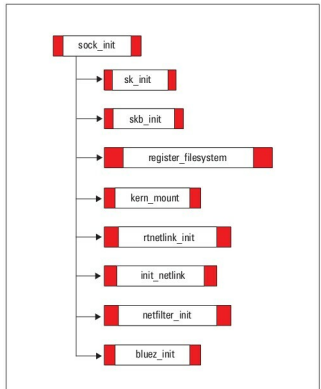


Figure 2: Function invocation tree for `sock_init()`